# SC4061 Computer Vision
# Lab 1: Point Processing, Spatial Filtering, Frequency Filtering and Imaging Geometry

Name: Gan Yee Hong
Matriculation No.: U2320111K

## 2.1 Contrast Stretching

### (a) Read image

```
Pc = imread('mrt-train.jpg');
whos Pc
P = rgb2gray(Pc);
```

Output:

```
Name        Size                Bytes  Class    Attributes

Pc        320x443x3            425280  uint8
```

- It has 3 channels, meaning RGB colour image, so it is converted to grayscale with rgb2gray.

### (b) View image

```
subplot(1,2,1); imshow(P); title('Original P');
```




Figure 1. Original mrt image

- The image (P) looks dull and greyish, with no true black or white, meaning poor contrast.

### (c) Intensity range

```
min(P(:)), max(P(:))
```

Output:

```
2.1 Original: min = 13, max = 204
```

- Only 75% of the 0–255 range is used (13-204), so the image looks dull (poor contrast).

### (d) Contrast stretching

```
P2 = imsubtract(P, double(min(P(:))));
P2 = immultiply(P2, 255 / double(max(P2(:))));
```

Output:

```
2.1 Stretched: min = 0, max = 255
```

- The image (P2) now uses the full 0–255 range.

### (e) Result

```
subplot(1,2,2); imshow(P2); title('Contrast stretched P2');
```

Output:




*Figure 2. Comparison with Original P (left) and Contrast-stretched P2 (right).*

- P2 has deeper blacks and brighter whites, and objects like trees and the train are more distinct.
- **Gan Yee Hong's comment:** imsubtract and immultiply keep P2 as uint8 throughout, rounding and clipping results to 0–255, so no double/uint8 conversion is needed before imshow.

## 2.2 Histogram Equalization

### (a) 10 vs 256 bins

```
subplot(3,2,1); imhist(P, 10);  title('P, 10 bins');
subplot(3,2,2); imhist(P, 256); title('P, 256 bins');
```

Output:

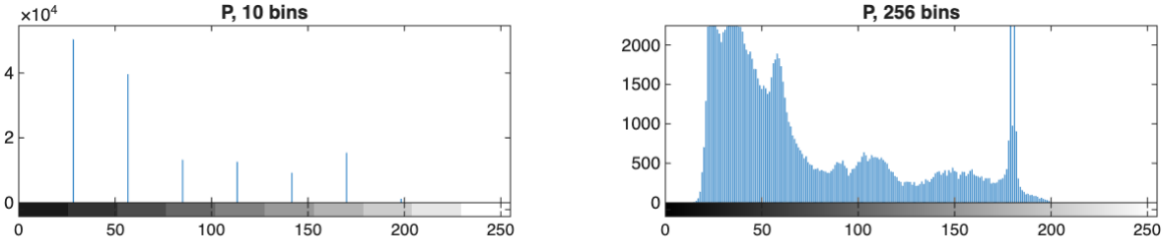



*Figure 3. Histograms with 10 and 256 bins(P)*

- **10 bins:** shows the coarse overview. Most pixels are dark (below 100).
- **256 bins:** shows every level. Values only occupy 13–204, with peaks around 30–60 and 180.

### (b) After histeq

```
P3 = histeq(P, 255);
subplot(3,2,3); imhist(P3, 10);  title('P3 = histeq(P), 10 bins');
subplot(3,2,4); imhist(P3, 256); title('P3 = histeq(P), 256 bins');
```

Output:

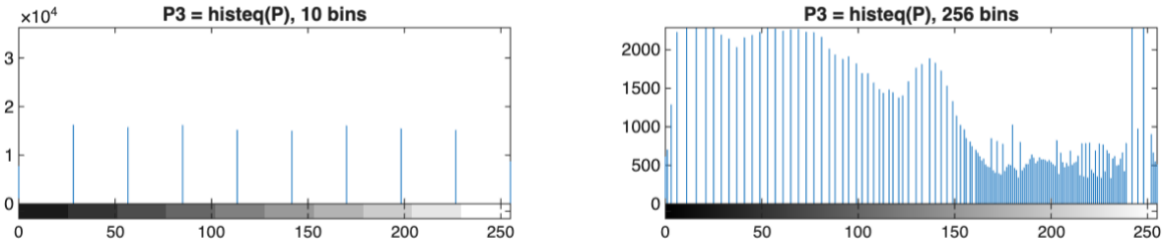



*Figure 4. Histograms with 10 and 256 bins after histeq(P3)*

- **10 bins:** approximately uniform (around 15000 pixels per bin).

- **256 bins:** not uniform. Tall and widely spaced bars where P was dense, while short and tightly packed bars where P was sparse. There are many empty levels in between.
- **Gan Yee Hong's comment on the differences:** the mapping is per gray level, so pixels with the same value cannot be split. Equalization only holds over ranges of intensity, which is what 10 bins measures.

### (c) Equalizing P3 again

```
P4 = histeq(P3, 255);
subplot(3,2,5); imhist(P4, 10);  title('P4 = histeq(P3), 10 bins');
subplot(3,2,6); imhist(P4, 256); title('P4 = histeq(P3), 256 bins');
```

Output:

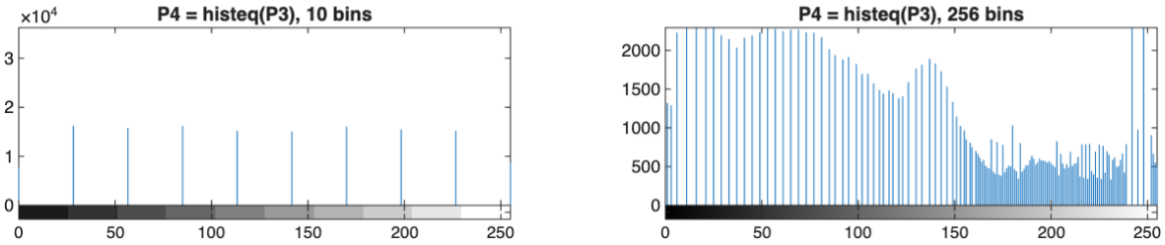



*Figure 5. Histograms with 10 and 256 bins after histeq again(P4)*

- **Result:** the histogram does not become more uniform; P4 is practically identical to P3.
- **Gan Yee Hong's comment on the reason:** P3 is already as flat as possible. histeq() can only move or merge whole gray levels, and never split them. Running histeq() again just merges one pair of levels due to rounding, which barely makes any difference.

## 2.3 Linear Spatial Filtering

### (a) Gaussian filters

```
[xx, yy] = meshgrid(-2:2, -2:2);
h1 = exp(-(xx.^2 + yy.^2) / (2*1^2)) / (2*pi*1^2);
h1 = h1 / sum(h1(:));
h2 = exp(-(xx.^2 + yy.^2) / (2*2^2)) / (2*pi*2^2);
h2 = h2 / sum(h2(:));

subplot(1,2,1); mesh(h1); title('Gaussian 5x5, \sigma = 1');
subplot(1,2,2); mesh(h2); title('Gaussian 5x5, \sigma = 2');
```

Output:

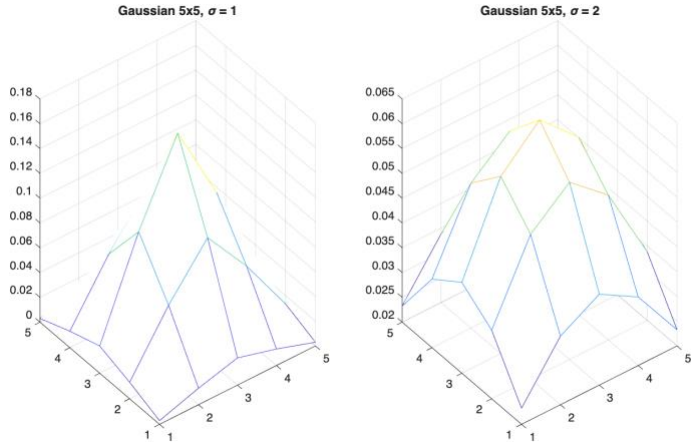



*Figure 6. Normalized 5×5 Gaussian filters, σ = 1 (left) and σ = 2 (right).*

- **Normalization:** both filters sum to 1, so overall brightness is preserved.

- **Shape:** σ = 1 is sharply peaked, while σ = 2 is much flatter.

**(b) & (c) Gaussian noise**

```
Pgn = read_gray('lib-gn.jpg');
Pgn_h1 = uint8(conv2(double(Pgn), h1, 'same'));
Pgn_h2 = uint8(conv2(double(Pgn), h2, 'same'));

subplot(1,3,1); imshow(Pgn);    title('lib-gn (Gaussian noise)');
subplot(1,3,2); imshow(Pgn_h1); title('Gaussian \sigma = 1');
subplot(1,3,3); imshow(Pgn_h2); title('Gaussian \sigma = 2');
```

Output:




*Figure 7. Gaussian noise: original(left), σ = 1(middle), σ = 2(right).*

- **Observation:** both filters clearly and effectively reduce the grain as zero-mean noise averages out.
- **σ = 1:** sharper, but more noise(grain) remains.
- **σ = 2:** removes more noise but more blurs edges and windows.
- **No filtering:** keeps all detail but all noise(grain).
- **Gan Yee Hong's comment**: This is a trade-off between noise removal and sharpness.

**(d)&(e) Speckle noise**

```
Psp = read_gray('lib-sp.jpg');
Psp_h1 = uint8(conv2(double(Psp), h1, 'same'));
Psp_h2 = uint8(conv2(double(Psp), h2, 'same'));

subplot(1,3,1); imshow(Psp);    title('lib-sp (speckle noise)');
subplot(1,3,2); imshow(Psp_h1); title('Gaussian \sigma = 1');
subplot(1,3,3); imshow(Psp_h2); title('Gaussian \sigma = 2');
```

Output:




*Figure 8. Speckle noise: original(left), σ = 1(middle), σ = 2(right).*

- **Observation:** the dots are dimmed and smeared into grey blobs, but are still not removed.
- **σ = 1**: sharper, but the dots remain clearly visible as small grey spots.
- **σ = 2**: dots are spread into fainter but larger blurry blobs, and edges and windows are more blurred.

- **Conclusion:** Gaussian filters handle Gaussian noise better than speckle noise.

1) Gaussian noise: every pixel has a small, zero-mean error (equally + or −), so averaging neighbours cancels the errors out.

2) Speckle noise: a few pixels are extreme outliers (near 0 or 255). One outlier pulls the weighted average of its whole neighbourhood, so the dot is spread into a blob instead of removed.

- **Gan Yee Hong's comment**: speckle needs a filter that ignores outliers rather than averaging them, which motivates the median filter in 2.4.

## 2.4 Median Filtering

```
Pgn_m3 = medfilt2(Pgn, [3 3]);
Pgn_m5 = medfilt2(Pgn, [5 5]);
Psp_m3 = medfilt2(Psp, [3 3]);
Psp_m5 = medfilt2(Psp, [5 5]);

subplot(1,3,1); imshow(Pgn);    title('lib-gn (Gaussian noise)');
subplot(1,3,2); imshow(Pgn_m3); title('Median 3x3');
subplot(1,3,3); imshow(Pgn_m5); title('Median 5x5');

subplot(1,3,1); imshow(Psp);    title('lib-sp (speckle noise)');
subplot(1,3,2); imshow(Psp_m3); title('Median 3x3');
subplot(1,3,3); imshow(Psp_m5); title('Median 5x5');
```

Output:




*Figure 9. Gaussian noise: original(left), median 3×3(middle), median 5×5(right).*




*Figure 10. Speckle noise: original(left), median 3×3(middle), median 5×5(right).*

- **Speckle noise:** median filtering removes almost all dots even at 3×3, since outliers are ignored rather than averaged. Much better than Gaussian filtering.
- **Gaussian noise:** median filtering reduces noise but less smoothly than Gaussian filtering; 5×5 gives a blotchy and painted look.
- **Edges:** median filtering keeps edges sharper, as it outputs an actual neighbouring value instead of blending across the edge.
- **Trade-offs:** larger window in median filtering means more noise removed but fine detail like thin lines and corners lost. Compared to Gaussian filter, median filtering is non-linear and slower as it does sort at every pixel.
- **Gan Yee Hong's comment:** Gaussian filter for Gaussian noise, median filter for speckle noise.

## 2.5 Suppressing Noise Interference Patterns

### (a) Interference image

```
Pi = imread('pck-int.jpg');  imshow(Pi);
```

Output:




*Figure 11. pck-int.jpg: interference patterns*

- Strong diagonal lines cover the whole image

### (b) Power spectrum and peak locations

```
F = fft2(double(Pi));
S = abs(F).^2;
imagesc(fftshift(S.^0.1)); colormap('default');
```

Output:

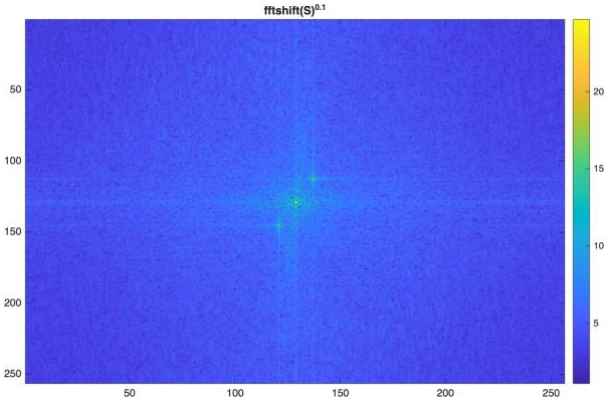



*Figure 12. Power spectrum S^0.1 with fftshift.*

- **Observation:** two bright peaks symmetric about the DC are the diagonal interference lines.

### (c) Power spectrum without fftshift and peak locations

```
imagesc(S.^0.1); colormap('default'); colorbar;
[px, py] = ginput(2);
peaks = round([px py]);
```

Output:

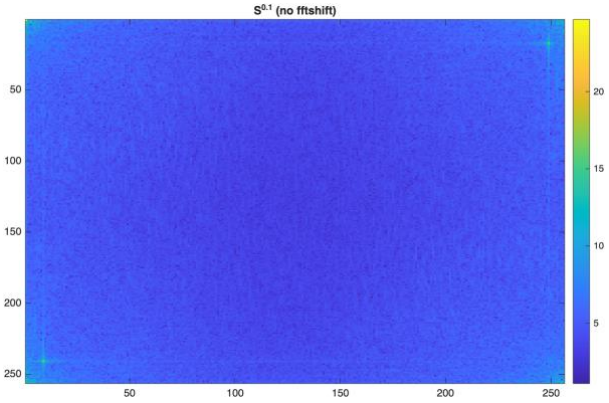



*Figure 13. Power spectrum S^0.1 without fftshift.*

```
2.5 Interference peaks (x, y): (9, 241) and (249, 17)
```

- **Peak locations: (9, 241) and (249, 17)**.

### (d) Zero 5×5 neighbourhoods in Fourier transform F

```
F2 = F;
for k = 1:2
    x = peaks(k,1);  y = peaks(k,2);
    F2(y-2:y+2, x-2:x+2) = 0;
end

S2 = abs(F2).^2;
imagesc(fftshift(S2.^0.1)); colormap('default'); colorbar;
```

Output:

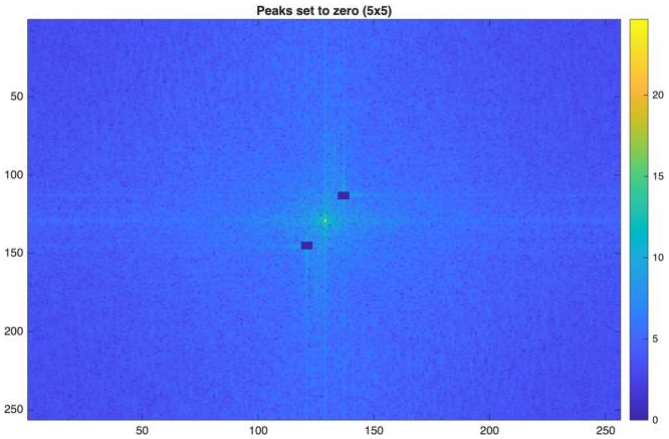



*Figure 14. Spectrum after zeroing the two peaks.*

### (e) Inverse FFT and improvement

```
Pi2 = real(ifft2(F2));


% improvement: also zero the row and column through each peak
F3 = F2;  F3(241,:) = 0;  F3(:,9) = 0;  F3(17,:) = 0;  F3(:,249) = 0;
F3(1,1) = F(1,1);           % keep DC (average brightness)
Pi3 = real(ifft2(F3));

subplot(1,3,1); imshow(Pi);                 title('Original');
subplot(1,3,2); imshow(uint8(Pi2));         title('5x5 peaks removed');
subplot(1,3,3); imshow(contrast_stretch(Pi3)); title('Peak rows/columns removed');
```

Output:




*Figure 15. Original(left), 5×5 peaks removed(middle), peak rows/columns also removed(right).*

- **Observation:** Diagonal lines are almost gone because each line pattern is a single frequency pair.
- **Issue identified:** faint lines remain because some interference energy lies around the peaks, along the horizontal and vertical lines through them (visible in Figure 14).

- **Improvement:** zeroing the full row and column through each peak removes most of the remaining lines, at the cost of slight loss of genuine content at those frequencies.

### (f) Freeing the primate

```
Fs = fftshift(fft2(double(Pp)));          % centred spectrum
d1 = [115 -49];
p2 = [9 -15];            % peak offsets measured from the spectrum
u = d1 / norm(d1);                        % direction of the 3 lines
for o = {c, c + p2, c - p2}               % one point on each line
    q = o{1};
    dist = abs(-(xx-q(1))*u(2) + (yy-q(2))*u(1));                % distance to line
    H = H .* (1 - exp(-dist.^2 / (2*w^2)) .* keep_centre);        % thin band = 0
end
Pp2 = real(ifft2(ifftshift(Fs .* H)));    % back to an image
```

Output:

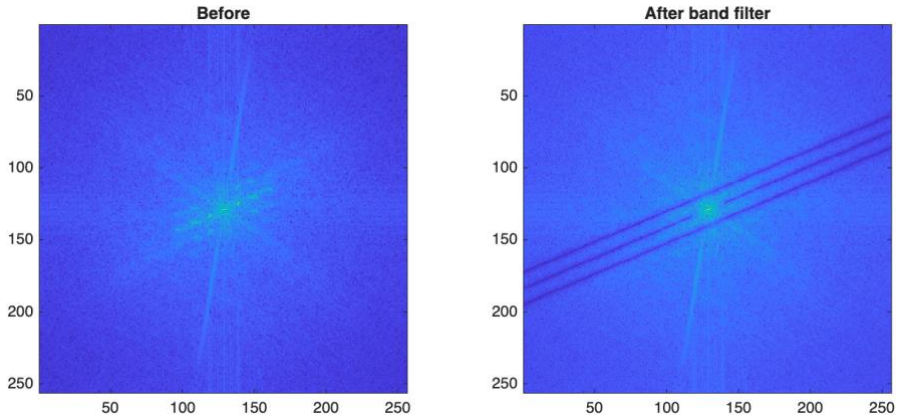



*Figure 16. Primate spectrum before (left) and after (right) band filter.*

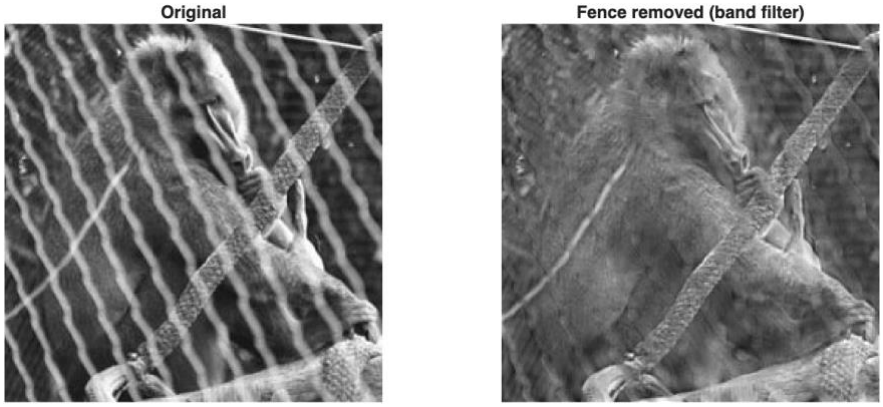



*Figure 17. Original(left), fence removed with band filter(right).*

- **Method:** the fence repeats regularly in 2D (a lattice), so its frequency peaks lie on 3 parallel lines in the spectrum (Figure 16, left).
- **Filter:** a thin band (half-width 2.5 pixels) along each line was set to 0, keeping the centre so the primate's overall shape is preserved (Figure 16, right).
- **Result:** most of the fence is removed and the primate's body and face are clearly visible.
- **Gan Yee Hong's comment:** it is still not perfectly clean. The thick rope and the bright diagonal wire do not repeat, so they are not on the lines and remain. But some primate detail at the removed frequencies is also lost.

## 2.6 Undoing Perspective Distortion

### (a) Book image

```
P = imread('book.jpg');  imshow(P);
```

Output:

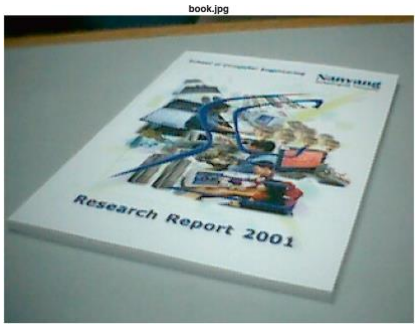



*Figure 18. Slanted view of an A4 book*

### (b) Corners of the book

```
[X, Y] = ginput(4);
x = [0; 210; 210;   0];
y = [0;   0; 297; 297];

for k = 1:4
    fprintf('  %-13s %-16s %s\n', corner_names{k}, ...
            sprintf('(%.0f, %.0f)', X(k), Y(k)), sprintf('(%d, %d)', x(k), y(k)));
end
```

Output:

```
2.6 Corners:
  Corner        Book (X, Y)      A4 target (x, y)
  Top-left      (143, 27)        (0, 0)
  Top-right     (308, 47)        (210, 0)
  Bottom-right  (256, 216)       (210, 297)
  Bottom-left   (4, 159)         (0, 297)
```

### (c) Projective transformation

```
for k = 1:4
    A(2*k-1,:) = [X(k) Y(k) 1 0 0 0 -x(k)*X(k) -x(k)*Y(k)];
    A(2*k,  :) = [0 0 0 X(k) Y(k) 1 -y(k)*X(k) -y(k)*Y(k)];
    v(2*k-1) = x(k);
    v(2*k)   = y(k);
end
u = A \ v;
U = reshape([u; 1], 3, 3)';
disp('2.6 Projective transformation matrix U:'); disp(U);
```

Output:

```
2.6 Projective transformation matrix U:
    1.4662    1.5425 -251.4688
   -0.4278    3.6897  -40.1170
    0.0002    0.0052    1.0000
```

```
w = U * [X'; Y'; ones(1,4)];
w = w ./ (ones(3,1) * w(3,:))
disp('2.6 Transformed corners (should be the A4 corners):'); disp(w);
```

Output:

```
2.6 Transformed corners (should be the A4 corners):
    0.0000  210.0000  210.0000   -0.0000
    0.0000   -0.0000  297.0000  297.0000
    1.0000    1.0000    1.0000    1.0000
```

- **Verification:** each column (one corner) is exactly (0,0), (210,0), (210,297), (0,297), which is the desired A4 corners. The transformation does give back the 4 corners of the desired image. Note: −0.0000 is rounding error.

**(d)&(e) Warped image**

```
T = maketform('projective', U');
P2 = imtransform(P, T, 'XData', [0 210], 'YData', [0 297]);
imshow(P2); title('Warped to frontal view (1 pixel = 1 mm)');
```

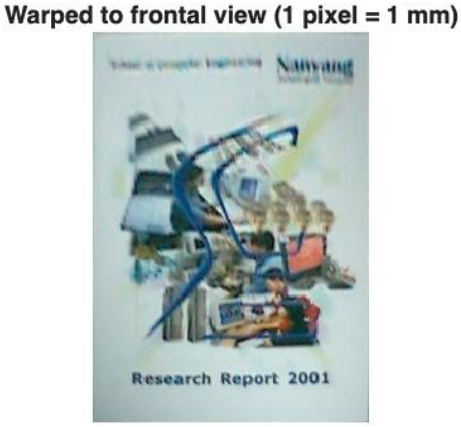



*Figure 19. Warped frontal view.*

- **Observation:** the book is now a frontal A4 rectangle at 1 pixel = 1 mm, which is expected.
- **Quality:** the top is blurry (the "School of Computer Engineering" text is unreadable), while the bottom is sharper(“Research Report 2001” is readable). On the top right, the “Nanyang” big word is still visible, while the words below it appear to be blurry. The middle image is slightly blurry as well.
- **Reasons:**
  1) The far (top and top left) end of the book covers few pixels in the original, so they are stretched by interpolation. Subtle issues like the low original resolution and small errors in the clicked corners may also contribute to this quality.
  2) The bottom (near) end covers many more pixels in the original, so it needs little stretching and stays sharp
  3) Large text like "Nanyang" also survives because its letters span several pixels even at the far end, while the small text below it does not.

**(f) Pink screen**

```
hsv = rgb2hsv(P2);
pink = (H > 0.85 | H < 0.05) & Sat > 0.25 & Val > 0.5;   % pinkish, vivid, bright
pink = bwareafilt(imopen(pink, strel('square',3)), 1);   % largest blob
bbox = regionprops(pink, 'BoundingBox').BoundingBox;
subplot(1,2,1); imshow(pink); title('Pink mask');
subplot(1,2,2); imshow(P2); hold on;
```

Output:




*Figure 20. Pink mask (left) and detected screen (right).*

```
2.6f Pink screen bounding box [x y width height] = [150.5 157.5 26.0 25.0]
```

- **Result:** identified by colour thresholding in HSV. The box is at x = 150.5, y = 157.5 mm, size 26 × 25 mm.

## 2.7 Two Perceptrons

- **Data:** $x(1) = [3\ 3\ 1]^T \in c1$, $x(2) = [1\ 1\ 1]^T \in c2$, $w(1) = [0\ 0\ 0]^T$. The last element 1 absorbs the bias into w.

**Algorithm 1: update only on misclassification (α = 1)**

```
alpha1 = 1;
w1 = [0; 0; 0];
if r(i) == 1 && w'*x <= 0,       w = w + alpha*x;   % 1') c1 misclassified
elseif r(i) == -1 && w'*x >= 0,  w = w - alpha*x;   % 2') c2 misclassified
end                                                 % 3') otherwise unchanged
```

Output:

```
2.7 Algorithm 1 iterations:
  w(2) = [3  3  1]
  w(3) = [2  2  0]
  w(4) = [2  2  0]
  w(5) = [1  1 -1]
  w(6) = [1  1 -1]
  w(7) = [0  0 -2]
  w(8) = [3  3 -1]
  w(9) = [2  2 -2]
  w(10) = [2  2 -2]
  w(11) = [1  1 -3]
  w(12) = [1  1 -3]
  w(13) = [1  1 -3]
2.7 Algorithm 1 result: w = [1  1 -3], boundary 1*x1 + 1*x2 + -3 = 0
```

- **Result:** $w = [1\ 1\ -3]^T$, boundary x1 + x2 − 3 = 0, which is the same as the lecture slides.

**Algorithm 2: gradient descent on $E(w) = \frac{1}{2}(r - w^T x)^2$ (α = 0.1, 100 iterations)**

```
alpha2 = 0.1;
n_iter = 100;
w2 = [0; 0; 0];
cost = zeros(1, n_iter);
for k = 1:n_iter
    i = mod(k - 1, size(Xp, 2)) + 1;
    x = Xp(:, i);
    w2 = w2 + alpha2 * (rp(i) - w2' * x) * x;
    cost(k) = mean(0.5 * (rp - w2' * Xp).^2);   % average cost over both samples
end
```

Output:

```
2.7 Algorithm 2 result: w = [0.4631     0.4631     -1.829], final cost = 0.0030
2.7 Algorithm 2 outputs w'x = [0.9499    -0.9026] (targets [1 -1])
```

- **Result:** w = [0.463 0.463 −1.829]$^T$, boundary 0.463x1 + 0.463x2 − 1.829 = 0, i.e. x1 + x2 − 3.95 = 0. Outputs are close to the targets +1 or -1, so the loss function is near 0.

```
subplot(1,3,1); plot_perceptron(Xp(1:2,:), rp, w1); title('Algorithm 1');
subplot(1,3,2); plot_perceptron(Xp(1:2,:), rp, w2); title('Algorithm 2');
subplot(1,3,3); plot(1:n_iter, cost, 'LineWidth', 1.5);
xlabel('Iterations'); ylabel('Cost'); title('Algorithm 2 cost'); grid on;
```

Output:

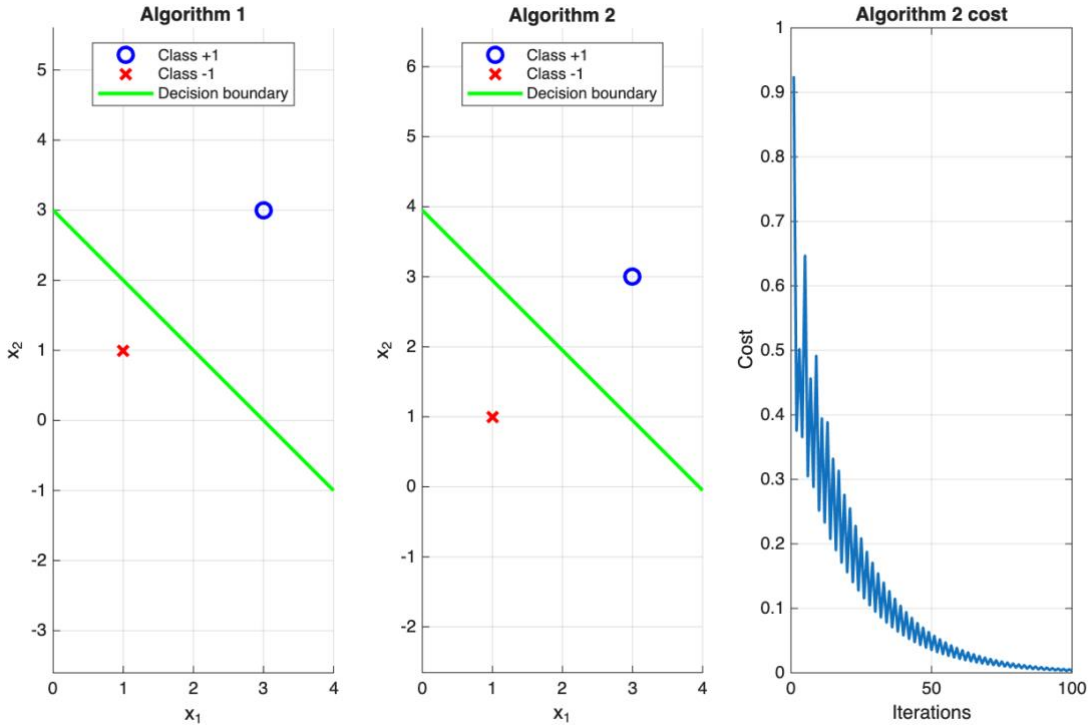



*Figure 21. Decision boundaries of Algorithms 1 and 2, and the cost curve of Algorithm 2.*

- **Comparison**

  1) **Same direction and gradient:** both boundaries are lines of the form x1 + x2 = constant.
  2) **Position:** Algorithm 1 stops at the first separating line (x1 + x2 = 3), which is closer to the c2 (class2) point. Algorithm 2 gives x1 + x2 ≈ 3.95, near the midpoint (2, 2), so it has a better margin.
  3) **Convergence:** Algorithm 1 converges after 10 iterations (7 updates) and stops once both points are correct. Algorithm 2 updates at every step, decreasing the cost gradually, and needs α to be chosen (0.1).

- **Gan Yee Hong's comment:** the cost curve zigzags because each update fits one of the two points in turn, slightly worsening the other.

## 2.8 SpectraPaint: paint on the spectrum

### (a) Motivation and problem

- **Motivation:** In 2.5, I had to find the noise peaks by hand, type their positions into the code and run it again every time I wanted to change. This was slow, and it was hard for me to understand what one point in the spectrum does to the image.
- **Problem:** Make the frequency domain interactive, i.e. edit the spectrum directly and see the image change instantly, and remove periodic noise automatically for any image.

- **Target users:** Students learning frequency-domain filtering, as in the first half of this course.

**(b) How the program was developed**

- **AI tool:** developed with the help of an AI tool (Anthropic Claude) for design ideas, code and testing.
- **Platform:** a single HTML file using plain JavaScript and HTML. There is nothing to install.
- **Own FFT:** a 2D radix-2 FFT for 256×256 images, verified against MATLAB fft2 (identical values; FFT then inverse FFT returns the image with an error of about $10^{-13}$).
- **Speed:** one full update (mask, inverse FFT, redraw) takes about 1 to 5 ms, so the image changes in real time while painting.
- **Steps:**
  1) I first wrote the FFT and filtering code, and I checked that its results match MATLAB's fft2.
  2) Next, I added automatic peak detection and tested it on pck-int, primate-caged and Lena. Lena has no periodic noise, so finding 0 peaks there showed the detection was not removing parts of a normal image by mistake.
  3) Last, I built the user interface and tested it in Chrome (Figures 22–26).

**(c) Features**

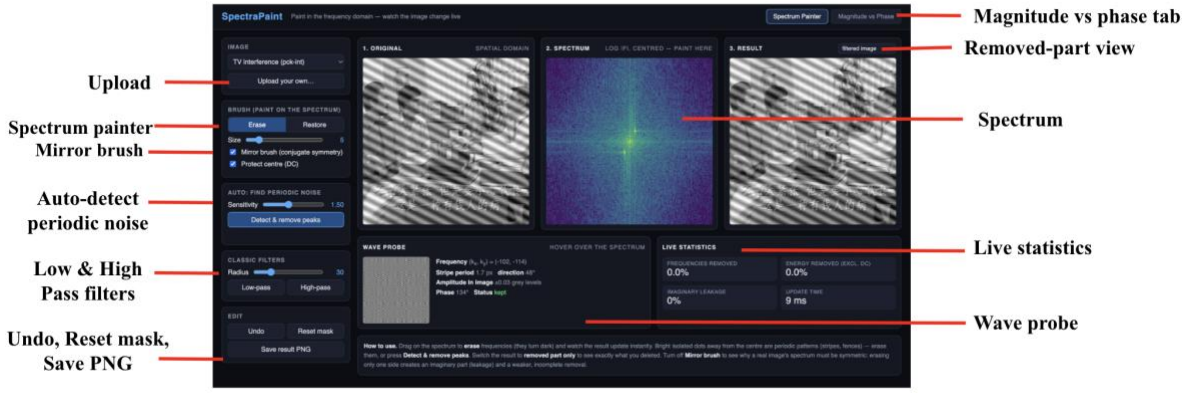



*Figure 22. Features of SpectraPaint.*

- **Spectrum painter:** erase or restore frequencies by painting on the spectrum, and the result updates in real-time.
- **Mirror brush:** also paints the mirrored point through the centre, because a real image has a conjugate-symmetric spectrum (the same reason both peaks are zeroed in 2.5).
- **Wave probe:** hovering over any spectrum point shows the exact stripe pattern it adds to the image, with its period, direction, amplitude (in grey levels) and phase.
- **Auto-detect periodic noise:** marks frequencies whose log-magnitude exceeds the local 15×15 average by a chosen sensitivity (default 1.5), away from the centre, and removes them in one click.
- **Removed-part view:** shows only what was deleted (original − result), to check that only the noise was removed.
- **Live statistics:** % of frequencies removed, % of energy removed, imaginary leakage and update time.
- **Magnitude vs phase tab:** swaps the magnitude and phase of two images.
- **Others:** low-pass and high-pass filters, undo, reset, upload your own image, save the result as PNG.

**(d) Useful results**

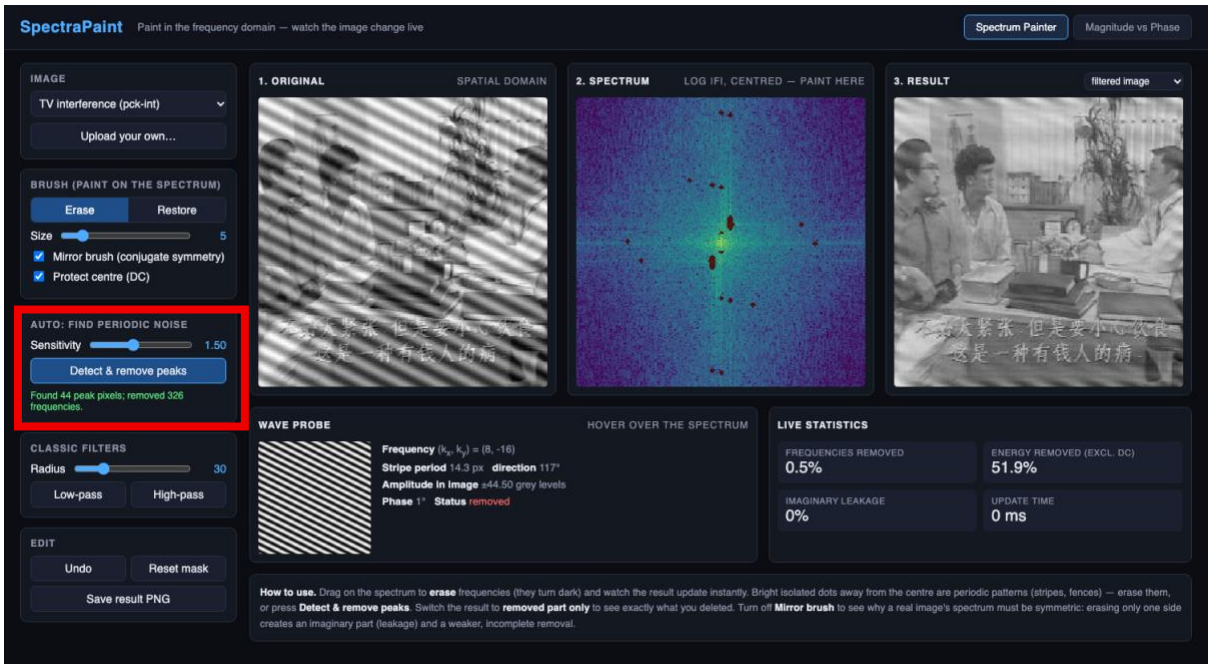



*Figure 23. Auto-detect on pck-int.jpg: the interference is removed in one click.*

- **Interference (pck-int):** one click finds 44 peak pixels and removes 326 frequencies (0.5%). The diagonal lines disappear completely, cleaner than the manual 5×5 notches in 2.5(e), and no peak measurement is needed.

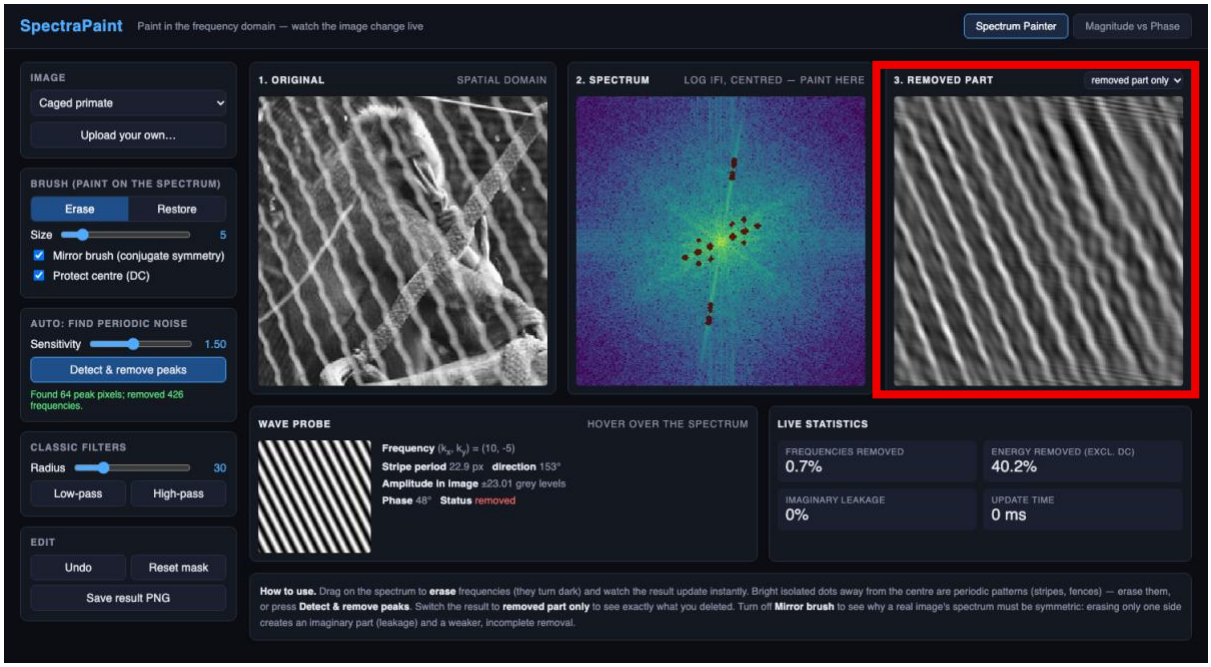



*Figure 24. Auto-detect on primate-caged.jpg, showing the removed part only.*

- **The removed part on Primate Image:** 64 peak pixels found and 426 frequencies (0.7%) removed. The removed-part view isolates the fence pattern almost perfectly, student can visualize the pattern removed.

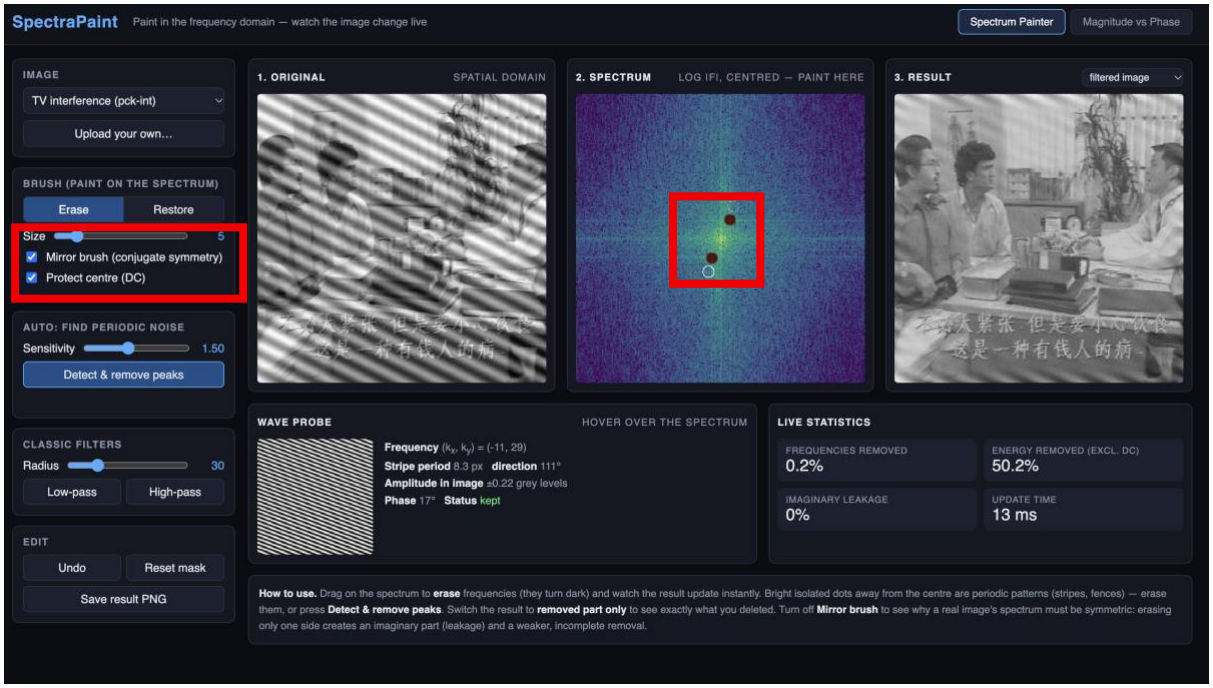



*Figure 25. Mirror brush on: erasing two interference peaks.*

- **Symmetry demo:** erasing only one of the two peaks reduces the stripes instead of removing them. A mirror brush enables a real image's spectrum to be edited symmetrically.

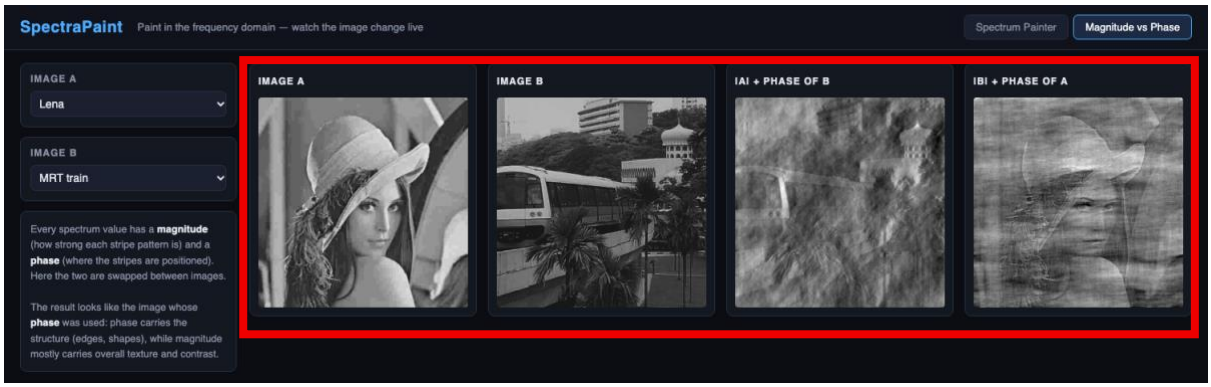



*Figure 26. Magnitude of one image combined with the phase of the other.*

- **Magnitude vs phase:** each result looks like the image whose phase was used, so phase carries the structure (edges, shapes) while magnitude mostly carries texture and contrast.

**(e) Program**

- **File:** 2.8-SpectraPaint.html, submitted with this report.
- **To run:** double-click to open it in any web browser (Chrome, Edge, Safari or Firefox). No installation or internet connection is needed.
- **Gan Yee Hong's comment:** the auto-detection reproduces and improves on the manual work in 2.5 in one click, and also works on any uploaded image.